

Quiz 2

601.471/671 NLP: Self-Supervised Models

Johns Hopkins University Fall 2026

Name: _____

Student version

Time: 15 minutes. Total: 100 points + 15 possible extra-credit points.

Complete Q1–Q4 for 100 points. Q5 is optional: leaving it blank does not reduce your score; earned bonus points are added beyond 100. The 15-minute limit includes any bonus work. For Q1, select exactly one answer per item; no explanation is needed. For Q2–Q5, show brief work. All logarithms are natural; equivalent exact expressions are accepted. No calculator is needed.

Q1. Training and Gradient Computation

25 points

- (a) (5 points) For one regression example, the residual $r = \hat{y} - y$ changes from 2 to 6. How do the losses r^2 (squared error) and $|r|$ (absolute error) change?
- ☐ Both losses are multiplied by 3.
 - ☐ Squared error is multiplied by 9; absolute error by 3.
 - ☐ Squared error is multiplied by 3; absolute error by 9.
 - ☐ Both losses are multiplied by 9.
- (b) (5 points) In an ordinary PyTorch training loop, which statement is correct?
- ☐ `loss.backward()` updates the parameter values.
 - ☐ `optimizer.step()` computes the derivatives of the current loss.
 - ☐ `loss.backward()` computes gradients; `optimizer.step()` updates parameters.
 - ☐ `optimizer.zero_grad()` resets all parameter values to zero.
- (c) (5 points) You compare four learning rates and choose the one with the lowest held-out loss. Which split should guide that choice?
- ☐ The validation set.
 - ☐ The test set, reused after every trial.
 - ☐ The training set, because it supplied the gradients.
 - ☐ Whichever split gives the lowest number.
- (d) (5 points) For a chain of L dense layers of fixed width, why does one backward sweep take $O(L)$ layer computations rather than $O(L^2)$?
- ☐ It approximates all hidden-layer gradients by zero.
 - ☐ It updates only the final layer.
 - ☐ It perturbs each parameter and reruns the entire network.
 - ☐ It reuses upstream gradients instead of repeating the work from later layers.
- (e) (5 points) A layer maps $x \in \mathbb{R}^3$ to $h = f(x) \in \mathbb{R}^5$. Define its Jacobian by $[J_f]_{ij} = \partial h_i / \partial x_j$. For a scalar loss $\ell(h)$, the upstream column gradient is $g_h = \nabla_h \ell \in \mathbb{R}^{5 \times 1}$. Which Jacobian shape and backward rule for $g_x = \nabla_x \ell$ are correct?
- ☐ $J_f : 3 \times 5; g_x = J_f g_h \in \mathbb{R}^{3 \times 1}$.
 - ☐ $J_f : 5 \times 3; g_x = J_f g_h \in \mathbb{R}^{5 \times 1}$.
 - ☐ $J_f : 5 \times 3; g_x = J_f^\top g_h \in \mathbb{R}^{3 \times 1}$.
 - ☐ $J_f : 3 \times 5; g_x = J_f^\top g_h \in \mathbb{R}^{5 \times 1}$.

Q2. Loss Reduction

15 points

For a batch of B labeled examples, the **mean cross-entropy loss** is

$$J_B(\theta) = -\frac{1}{B} \sum_{i=1}^B \ln p_{\theta}(t_i | x_i),$$

where θ denotes the trainable parameters, x_i is input i , t_i is its correct class, and $p_{\theta}(t_i | x_i)$ is the probability assigned to that class. For the original batch, $B = 2$ and these probabilities are $1/4$ and $1/2$.

- (a) (5 points) Compute the **mean** cross-entropy over the two examples.

Your answer:

- (b) (10 points) Duplicate **each** example once, making a batch of four. At the same parameter values, does (i) the mean loss change, and does (ii) its gradient with respect to the trainable parameters change? Explain each answer (5 points each). Assume each per-example loss depends only on that example and the parameters, with no randomness or additional loss terms. No numerical gradient is requested.

Your answer:

Q3. A Small Network: Backward Pass and Update

40 points

For one example, consider a network with one sigmoid hidden neuron and a scalar output:

$$z = w^\top x + b, \quad h = \sigma(z), \quad \hat{y} = uh, \quad \ell = \frac{1}{2}(\hat{y} - t)^2.$$

The input x and target t are fixed. The trainable parameters are $w \in \mathbb{R}^2$, $b \in \mathbb{R}$, and $u \in \mathbb{R}$; there is no output bias. Use column-vector gradients and the values

$$x = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad w = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad b = 0, \quad u = 2, \quad t = 0.$$

Recall that sigmoid is the nonlinear activation $\sigma(z) = 1/(1 + e^{-z})$, with $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Evaluate all derivatives in (b)–(c) at the original values above, before any update.

- (a) (10 points) Compute z , h , $\hat{y}$, and ℓ (2.5 points each).

Your answer:

- (b) (10 points) Compute $\partial\ell/\partial\hat{y}$ and $\partial\ell/\partial z$ (5 points each). Show the chain-rule factors for the latter.

Your answer:

- (c) (15 points) Compute all three parameter gradients: $\nabla_w \ell$, $\partial\ell/\partial b$, and $\partial\ell/\partial u$ (5 points each).

Your answer:

- (d) (5 points) Using the gradient from the original forward pass, compute the new vector w' after one gradient-descent step with learning rate $\eta = 0.1$. You do not need to compute the updated b , u , or loss.

Your answer:

Q4. Reading Autograd Code

20 points

Consider this complete snippet. Each `backward()` call uses a freshly computed expression. The method `.item()` returns a scalar tensor's numerical value.

```
import torch

w = torch.tensor(-1.0, requires_grad=True)
(w ** 3).backward()
print(w.grad.item())      # A
(4 * w).backward()
print(w.grad.item())      # B
print(w.item())           # C
```

- (a) (15 points) What numbers are printed at A, B, and C in the original code (5 points each)?

Your answer:

- (b) (5 points) Insert one line immediately before `(4 * w).backward()` so B reports only the derivative of $4w$ with respect to w . Give the inserted line (3 points) and explain why it is needed (2 points).

Your answer:

Q5. Optional Bonus: One Input, Two Paths

15 bonus points

This question is optional extra credit. Let

$$a = xy, \quad b = x + y, \quad F = ab,$$

Evaluate at $x = 2$ and $y = 1$. First find a , b , $\partial F/\partial a$, and $\partial F/\partial b$ (3 points total). Then compute $\partial F/\partial x$ and $\partial F/\partial y$ by backpropagation (6 points each). For each input derivative, **show the contribution through a and through b separately** before adding them. No graph drawing is required.

Your answer: